

Proyecto I: Informe de Resultados

Juan P. Elizondo, Estudiante TEC y Jose E. Tencio, Estudiante TEC

ÍNDICE

I.	Arquitectura general del ambiente	3
I-A.	Componentes principales	3
I-B.	Paquetes de comunicación	4
II.	Pruebas aplicadas por el ambiente	8
II-A.	Escenarios dirigidos	8
II-B.	Escenarios aleatorios	9
III.	Cómo correr el ambiente	10
III-A.	Acceso al servidor	10
III-B.	Lugar de trabajo	10
III-C.	Manejo del repositorio	10
III-D.	Carga de la herramienta (VCS)	11
III-E.	Proceso de compilación	11
III-F.	Proceso de corrida	12
III-G.	Herramienta de limpieza	13
IV.	Resultados de las pruebas aplicadas	13
IV-A.	Configuración de la corrida	13
IV-B.	Reporte final del checker	13
IV-C.	Análisis de retardos e histograma	15
IV-D.	Observaciones sobre los escenarios dirigidos	16
IV-E.	Observaciones sobre el escenario aleatorio	16

I. ARQUITECTURA GENERAL DEL AMBIENTE

I-A. Componentes principales

El ambiente desarrollado se basa en una arquitectura de ambiente de verificación en capas, de la cual se excluyen las aserciones y la cobertura funcional. Los transactores principales incluyen al Generador, Agente, Scoreboard, Checker, Monitor y Driver; estos dos últimos, fueron expandidos al considerar la necesidad de enrutar el tráfico correctamente por los cuatro dispositivos (controladores) del DUT, de la siguiente manera:

- **Driver:** Este se estructura en dos clases: `Driver_controller`, que actúa como clase padre, y `Driver_son`, que actúa como clase hijo. Mientras que el controlador del Driver se instancia una sola vez y es el que tiene comunicación con el resto del Ambiente, la clase `Driver_son` se instancia en 4 ocasiones (una para cada dispositivo).
- **Monitor:** El monitor igualmente está estructurado en dos clases: `Monitor_controller` y `Monitor_son`, las cuales actúan como clase padre y clase hijo del monitor, respectivamente. Al igual que en el Driver, el controlador del Monitor es el único que se comunica con el ambiente, esto significa que su clase hijo no tiene comunicación directa con el ambiente, ni tampoco se llega a instanciar dentro del ambiente como los demás transactores.

Por su parte, la interfaz que hace posible la conexión con el DUT delimita el acceso a sus puertos por medio de *modports*. Según se muestran a continuación los tres *modports* presentes.

```
modport driver_mp (  
    input  clk, pop, push, D_push,  
    output reset, pndng, D_pop  
);
```

```
modport monitor_mp(  
    input  clk, reset, pop, push, D_push, D_pop, pndng  
);
```

```
modport dut_mp(  
    input clk, reset, pndng, D_pop,  
    output pop, push, D_push  
);
```

Tal como se observa en el diagrama de la arquitectura del ambiente (Figura 1), existen diferentes *mailboxes* para la comunicación entre los transactores, cada uno de ellos se define para una clase de paquete de comunicación específico. Esta información se detalla mejor en la siguiente sección.

I-B. Paquetes de comunicación

El ambiente cuenta con tres clases de paquetes de comunicación distintos, ordenándolos desde el menor nivel hasta más alto nivel están los siguientes.

- **trans_bus.sv:** Esta es la clase de paquete de más bajo nivel dentro del ambiente, lo cual significa que maneja señales puramente binarias con diferentes tamaños. La adoptan 6 *mailboxes* según se muestra en la Figura 1.

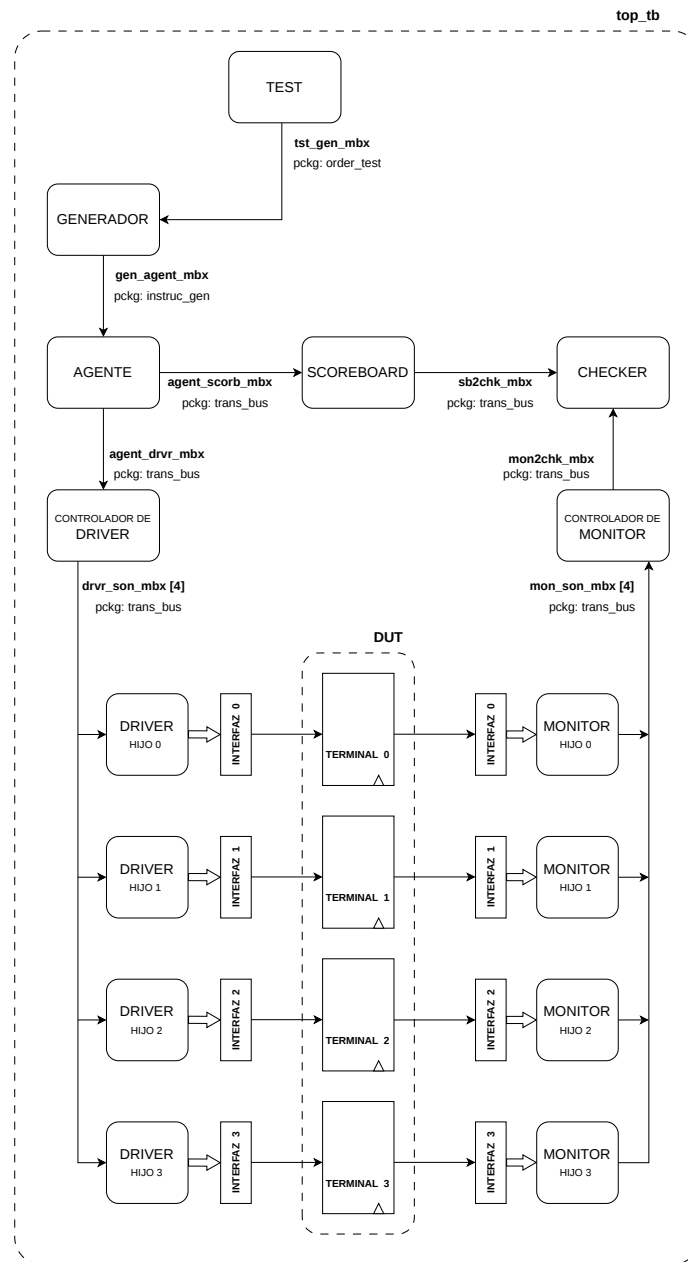


Figura 1. Esquema del Ambiente de Verificación.

Cuadro I
ATRIBUTOS DE LA CLASE TRANS_BUS.SV

Atributo	Tipo y/o tamaño	Descripción
id_origen	rand bit [7 : 0]	Indica la terminal que debe de emitir la transacción.
id_destino	rand bit [7 : 0]	Indica la terminal que es destinataria de la transacción.
payload	rand bit [pckg_sz-9 : 0]	Es el espacio de información relevante dentro de la transacción, excluyendo los metadatos.
delay	rand int unsigned	Corresponde a la cantidad de ciclos de espera entre paquetes.
id	int unsigned	Corresponde al identificador de la transacción.
n_created	static int unsigned	Opción para llevar un conteo de la cantidad de transacciones.
cfg	bus_config	Es el puntero al archivo de configuración.
mon_kind	mon_event_e	Es utilizado por el monitor/checker para su funcionamiento.
device_id	int unsigned	Es utilizado por el monitor/checker para su funcionamiento.
t_event	real	Tiempo de la transacción.

- **instruc_gen.sv:** Esta clase de paquete es de un nivel medio/alto, en la que se manejan principalmente tres tipos de transacciones básicas que el Generador le indica al Agente, las cuales son `trans_aleatoria`, `trans_dirigida` y `trans_secuencial`. La primera de ellas, se propuso como una forma de generar una única transacción aleatoria; no obstante, en el Test no se llegó a implementar debido a que era redundante con `trans_secuencial` la cual genera n cantidad de transacciones aleatorias, aún así se mantuvo como una de las capacidades de esta clase de paquete y del Agente. Adicionalmente, la clase cuenta con otros atributos según se muestra en el Cuadro II.

Cuadro II
ATRIBUTOS DE LA CLASE INSTRUCC_GEN.SV

Atributo	Tipo y/o tamaño	Descripción
tipo	instruc_tipo_e	Corresponde a uno de los tres tipos básicos de instrucciones.
id_destino	bit [7 : 0]	Le indica al Agente el identificador de destino en caso de que no se deba de aleatorizar.
id_origen	bit [7 : 0]	Le indica al Agente el identificador de origen en caso de que no se deba de aleatorizar.
payload	bit [pkg_sz-9 : 0]	Le indica al Agente el payload de la transacción en caso de que no se deba de aleatorizar.
delay	int unsigned	Le indica al Agente el retardo de la transacción en caso de que no se deba de aleatorizar.
set_id_destino	bit	Señal de control para indicar que si debe de aleatorizar este atributo (1b'0) o no (1b'1).
set_id_origen	bit	Señal de control para indicar que si debe de aleatorizar este atributo (1b'0) o no (1b'1).
set_payload	bit	Señal de control para indicar que si debe de aleatorizar este atributo (1b'0) o no (1b'1).
set_delay	bit	Señal de control para indicar que si debe de aleatorizar este atributo (1b'0) o no (1b'1).
cantidad	int unsigned	Indica la cantidad de transacciones que se deben de crear durante una secuencia aleatoria.

- **order_test.sv:** Esta es la clase de paquete de más alto nivel, en el que la abstracción llega a tal punto en el que el Test solamente le dice al Generador el escenario de pruebas que quiere llevar a cabo. Su atributo principal como es de esperar, corresponde al tipo de escenario a probar los cuales son `scen_aleatorio_sec`, `scen_aleatorio`, `scen_arbitraje_simultaneo`, `scen_broadcast`, `scen_invalido`, `scen_dispos` y `scen_autodirec`. Además de otros atributos relevantes según se muestran en el Cuadro III.

Cuadro III
ATRIBUTOS DE LA CLASE ORDER_TEST.SV

Atributo	Tipo y/o tamaño	Descripción
tipo	orden_tipo_e	Le indica al Generador cuál de los 7 tipos de escenarios quiere llevar a cabo.
cantidad	int unsigned	Aplica para transacciones secuenciales.
delay_secuencia	int unsigned	Aplica para el delay en transacciones secuenciales.
terminal_origen	bit [7 : 0]	Indica la terminar que debe de ser la emisora del escenario.

II. PRUEBAS APLICADAS POR EL AMBIENTE

El Test define la secuencia de escenarios que se inyectan al ambiente mediante órdenes de alto nivel. Cada escenario se construye como un objeto `order_test`, se publica en el buzón `env.tst_gen_mbx` y es traducido por el Generador a instrucciones `instruc_gen`; posteriormente, el Agente las convierte en transacciones `trans_bus` y el Driver las lleva al DUT. Esta separación permite que el Test describa el comportamiento deseado sin conocer los detalles de temporización, arbitraje o codificación de la interfaz.

La corrida de ejemplo ejecuta cinco escenarios dirigidos y un escenario aleatorio. Los escenarios dirigidos se concentran en condiciones específicas de arbitraje, difusión, direccionamiento inválido, apoderamiento del bus y autodireccionamiento. El escenario aleatorio corresponde a una secuencia tipo *soak*, en la que se mezclan destinos válidos, inválidos, broadcast, retardos y tamaños de carga útil.

II-A. Escenarios dirigidos

Los escenarios dirigidos se caracterizan porque el Test fija explícitamente los parámetros relevantes de cada caso a través de los atributos de `order_test`, tales como `terminal_origen`, `cantidad` o el propio `tipo` de escenario. De esta forma, el Generador no aleatoriza el comportamiento global del escenario, sino que lo reproduce de manera determinista según lo solicitado.

II-A0a. Arbitraje simultáneo.: La tarea `correr_arbitraje_simultaneo()` publica una orden con `tipo = scen_arbitraje_simultaneo`. El Generador interpreta esta orden como una solicitud de tráfico concurrente desde las cuatro terminales, de modo que todas ellas compitan por el bus en la misma ventana de arbitraje. El propósito de este escenario es ejercitar la lógica de arbitraje del DUT frente a solicitudes simultáneas y verificar que el ambiente es capaz de mantener los cuatro

canales activos sin perder transacciones ni corromper su contenido. Al no fijarse un origen particular, el escenario es simétrico respecto a los cuatro dispositivos.

II-A0b. Broadcast.: La tarea `correr_broadcast()` emplea `tipo = scen_broadcast` y asigna `terminal_origen = cfg.broadcast_origen`. En este escenario, una única terminal emite una transacción cuyo destino corresponde a todas las demás terminales del sistema. El objetivo es verificar que el DUT replica correctamente la transacción hacia los receptores y que el monitor/checker registra una TX y múltiples RX asociadas al mismo paquete. El origen del broadcast se parametriza mediante `broadcast_origen`, lo que permite reutilizar el escenario con distintas terminales emisoras sin modificar el Test.

II-A0c. Dirección inválida.: La tarea `correr_direccion_invalida()` utiliza `tipo = scen_invalido`. En este caso, los cuatro terminales emiten transacciones dirigidas a un identificador de destino que no corresponde a ninguna terminal válida del sistema. El escenario pretende confirmar que el DUT no genera recepciones espurias ante direcciones inválidas y que el ambiente no se bloquea ni produce transacciones fantasma. Además, este escenario verifica que el checker clasifica adecuadamente las transacciones no entregables, distinguiéndolas de las P2P, broadcast y de autoenvío.

II-A0d. Terminal acaparador.: La tarea `correr_terminal_acaparador()` utiliza `tipo = scen_dispSos` y asigna `terminal_origen = cfg.acaparador_origen` junto con `cantidad = cfg.acaparador_cantidad`. El escenario modela una terminal que solicita el bus de manera consecutiva un número determinado de veces. Su propósito es verificar que el árbitro del DUT no queda bloqueado ante ráfagas originadas por un mismo dispositivo y que el ambiente es capaz de encolar varias instrucciones consecutivas desde el mismo `Driver_son` sin perderlas. La cantidad de solicitudes consecutivas se controla mediante el atributo `cantidad` de la orden.

II-A0e. Autoenvío.: La tarea `correr_autoenvio()` emplea `tipo = scen_autodirec` y `terminal_origen = cfg.autoenvio_origen`. El escenario consiste en que una terminal se envía una transacción a sí misma, es decir, el identificador de origen coincide con el de destino. El objetivo es validar el tratamiento del autodireccionamiento dentro del DUT y comprobar que el checker es capaz de distinguir este caso de una transferencia punto a punto convencional. El origen del autoenvío se parametriza mediante `autoenvio_origen`.

II-B. Escenarios aleatorios

La tarea `correr_aleatorio()` utiliza `tipo = scen_aleatorio_sec`, con `cantidad = cfg.n_soak` y `delay_secuencia = cfg.soak_delay`. A diferencia de los escenarios anteriores, aquí el Test no fija origen, destino, payload ni retardo de cada transacción. En su lugar, delega en el

Generador la creación de una secuencia de N transacciones aleatorias, cuyo comportamiento global queda determinado por los pesos `wt_destino_valido`, `wt_destino_invalido` y `wt_broadcast`, así como por los rangos `min_delay/max_delay` y `payload_min/payload_max`. Este escenario constituye el único caso no dirigido dentro del Test y su propósito es someter al DUT y al ambiente a una mezcla heterogénea de tráfico que combine todos los casos anteriores.

III. CÓMO CORRER EL AMBIENTE

Para correr el ambiente y aplicar las pruebas se utiliza el servidor de la Escuela de Ingeniería Electrónica, siguiendo la siguiente serie de pasos.

III-A. Acceso al servidor

Para poder acceder al servidor se puede hacer de dos maneras. La primera mediante una terminal de Linux ejecutando un SSH a la dirección IP `172.21.8.7` (se necesita estar conectado a la red de la Escuela de Ingeniería ELectrónica), o bien utilizando el acceso mediante Omnisia Horizon Client utilizando la herramienta PuTTY de igual forma a la dirección IP `172.21.8.7`.

En ambos métodos el servidor va a solicitar iniciar sesión, para lo cual se debe de utilizar el nombre de usuario brindado por el profesor del curso, junto con la contraseña de usuario previamente configurada.

III-B. Lugar de trabajo

Algo importante a tener en consideración es que no se debe de trabajar en el home directamente luego de iniciar sesión en el servidor. Por el contrario, debe de trabajarse en la carpeta designada según cada usuario, por ejemplo:

```
cd /mnt/vol_NFS_rh003/Est_Veri_II2026/  
ELIZONDO_ESPINOZA_JUAN_VeriII26
```

Una alternativa, es utilizar un alias que guarde el comando anterior para facilitar el acceso a la carpeta de trabajo.

III-C. Manejo del repositorio

El repositorio de trabajo para este proyecto se puede acceder en el siguiente enlace: **VFCI-P1**

Una vez en la carpeta de trabajo, se debe de clonar el repositorio en el que se encuentra el ambiente en caso de que no haya sido clonado previamente. Para este caso, el comando junto con el enlace del repositorio es el siguiente:

```
git clone https://github.com/tencioeduardo/VFCI-Proyecto-1.git
```

Una vez que el repositorio haya sido clonado y se encuentre en la dirección de carpetas correcta (dentro del repositorio local), se debe de considerar la siguiente jerarquía de carpetas del repositorio:

```
VFCI-Proyecto-1/  
  doc/  
  src/  
  tb/  
  sim/  
  README.md  
  .gitignore
```

para utilizar el Makefile y así facilitar la compilación y corrida del ambiente, se debe de dirigir a la carpeta `sim` mediante el siguiente comando:

```
cd sim
```

III-D. Carga de la herramienta (VCS)

Antes de iniciar con las pruebas, es necesario ejecutar un comando para cargar la herramienta VCS de Synopsys y que los siguientes comando de compilación y corrida sean reconocidos, en la terminal:

```
source /mnt/vol_NFS_rh003/estudiantes/archivos_config/  
synopsys_tools2.sh
```

III-E. Proceso de compilación

En la carpeta `sim`, se encuentra el Makefile que describe la manera de compilación del código, para lo cual ya se encuentra descrito un comando de compilación, que se ejecuta con escribir lo siguiente en la terminal:

```
make comp
```

Esto compila el hardware y el testbench una sola vez generando el archivo binario `simv`.

III-F. *Proceso de corrida*

En el Makefile también se encuentra descrito la manera en la que se lleva a cabo la corrida de la simulación, se logra escribiendo lo siguiente en la terminal:

```
make run
```

Adicionalmente, para este comando existe la opción de utilizar `plusargs`, con los que se puede modificar lo siguiente:

Parámetros de configuración general del test

SEED

TIMEOUT

DRAIN_CYCLES

PCKG_SZ (necesita recompilar)

Parámetros de campos aleatorizables

WT_DESTINO_VALIDO

WT_DESTINO_INVALIDO

WT_BROADCAST

MIN_DELAY

MAX_DELAY

PAYLOAD_MIN

PAYLOAD_MAX

Parámetros para escenario aleatorio

N_SOAK

SOAK_DELAY

Parámetros de escenarios dirigidos del test

BROADCAST_ORIGEN

ACAPARADOR_ORIGEN

AUTOENVIO_ORIGEN

ACAPARADOR_CANTIDAD

Al momento de ejecutar el comando `run`, es posible especificar valores para los parámetros del

recuadro anterior, ya sea un único parámetro o varios. Por ejemplo:

```
make run BROADCAST_ORIGEN=3 N_SOAK=75 PAYLOAD_MAX=217
```

La semilla especialmente, está configurada como `automatic`, por lo que si no se especifica un valor concreto, se utilizará una semilla aleatoria cada vez que se ejecute `run`.

III-G. Herramienta de limpieza

Adicionalmente, el `Makefile` cuenta con otra opción de comando que permite hacer una limpieza de los archivos temporales y logs generados durante la compilación y ejecución (almacenados en la carpeta `sim`), ejecutando:

```
make clean
```

IV. RESULTADOS DE LAS PRUEBAS APLICADAS

IV-A. Configuración de la corrida

La corrida de ejemplo se ejecutó con semilla automática, la cual resultó en 1231381905. Los parámetros principales de la simulación fueron: `timeout=10000`, `drain_cycles=500`, `drvrs=4`, `wt_destino_valido=70`, `wt_destino_invalido=20`, `wt_broadcast=10`, `min_delay=0`, `max_delay=5`, `payload_min=0`, `payload_max=255`, `broadcast_origen=0`, `acaparador_origen=1`, `autoenvio_origen=2`, `acaparador_cantidad=7`, `n_soak=15` y `soak_delay=2`. El reporte CSV se generó en `packets.csv` y el histograma de retardos en `delay_histogram.png`.

La simulación finaliza con el mensaje `>> TEST PASSED <<` en $T = 11\,595\,000$ ps, con un tiempo de CPU de 0,990 s. El drenado de las colas TX se completa en $T = 6\,595\,000$ ps, es decir, antes del fin de la simulación, lo que confirma que el mecanismo `wait_empty` opera correctamente. El *watchdog* no se activa, por lo que no se alcanza el límite de `timeout`.

IV-B. Reporte final del checker

El reporte final del checker se resume en el Cuadro IV. Se observan 32 transacciones esperadas y 32 TX observadas, lo que indica que todas las transacciones planificadas llegaron al DUT. Las RX totales observadas son 23, coincidentes con las filas escritas en el CSV. No se reportan *mismatches* de payload, transacciones faltantes, RX inesperadas, TX inesperadas, coincidencias ambiguas ni retardos negativos.

Cuadro IV
 REPORTE FINAL DEL CHECKER PARA LA CORRIDA DE EJEMPLO.

Métrica	Valor	Observación
Esperados totales	32	Transacciones planificadas
TX totales observadas	32	Todas las TX esperadas
RX totales observadas	23	Recepciones válidas registradas
Resets observados	0	Sin reset durante el tráfico
Matches P2P	17	Punto a punto
Matches Autoenvío	7	Según etiqueta del checker
Matches Broadcast	2	Transacciones broadcast
Matches Inválido	6	Direcciones inválidas
Mismatches payload	0	Sin errores de datos
Missing TX / RX	0 / 0	Sin pérdidas
Unexpected TX / RX	0 / 0	Sin tráfico inesperado
CSV filas escritas	23	Coincide con RX observadas
CSV matches ambiguos	0	Sin ambigüedad
CSV delays negativos	0	Retardos consistentes

La diferencia entre las 32 TX esperadas y las 23 RX observadas se explica por la naturaleza de los escenarios. Las coincidencias de autoenvío (7) e inválido (6) se verifican a nivel de TX y de comportamiento, pero no generan una fila RX en el CSV. Por otro lado, cada transacción broadcast produce tres recepciones, por lo que las 2 TX de broadcast aportan 6 filas RX. Así, $32 - 7 - 6 + 4 = 23$, lo cual es consistente con las 23 filas del CSV.

Cuadro V
RESUMEN DE COINCIDENCIAS DEL CHECKER FRENTE A LAS FILAS DEL CSV.

Tipo	Matches checker	Filas CSV	Comentario
P2P	17	17	Una RX por TX
Broadcast	2	6	2 TX, 3 RX por TX
Autoenvío	7	0	Verificación sin RX en CSV
Inválido	6	0	TX sin recepción válida
Total	32	23	32 TX, 23 RX

IV-C. Análisis de retardos e histograma

A partir de `packets.csv` se generó el histograma de retardos mediante `make plot`, el cual ejecuta `gnuplot` con el script `scripts/histogram.gp`. El histograma utiliza un ancho de bin de 10 ns y resume las 23 recepciones válidas. La media de retardo es 520,87 ns y el máximo es 790,00 ns.

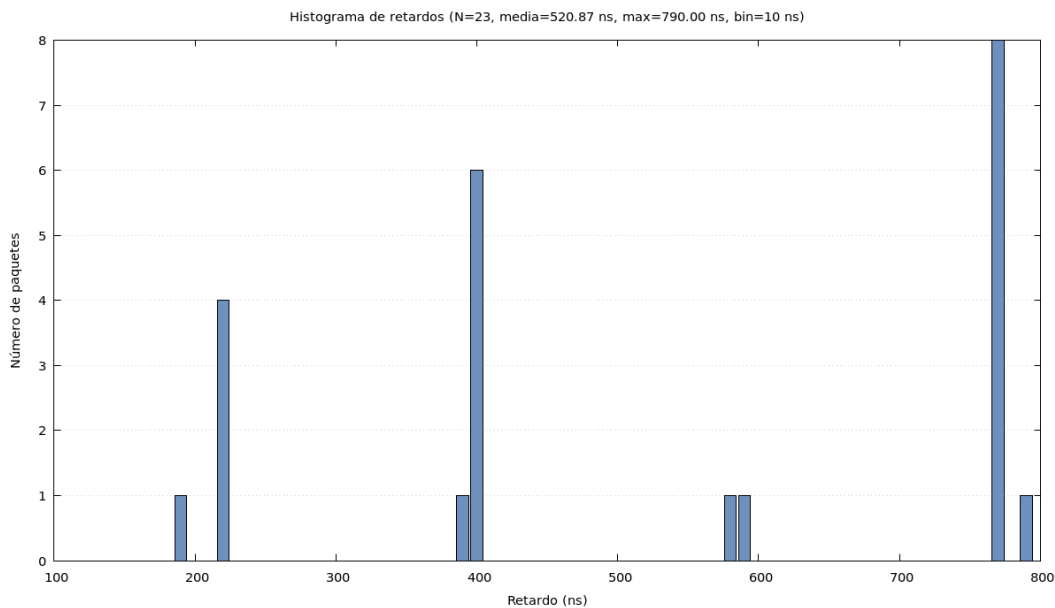


Figura 2. Histograma de retardos de las 23 recepciones registradas en `packets.csv`. La media es 520,87 ns y el máximo 790,00 ns.

En la Figura 2 se observan tres concentraciones principales de retardo: una alrededor de 190–220 ns, otra cercana a 400 ns y una tercera entre 770 y 790 ns. Estos grupos reflejan diferentes condiciones de

tráfico y niveles de contención en el bus. Los retardos más bajos se asocian a transacciones con poca competencia, mientras que los más altos aparecen en escenarios con broadcast, ráfagas desde un mismo terminal o mayor ocupación del bus. La ausencia de retardos negativos y de valores atípicos fuera del rango observado respalda la consistencia temporal del monitor y del checker.

IV-D. Observaciones sobre los escenarios dirigidos

Los escenarios dirigidos cubren las condiciones funcionales más relevantes del protocolo. El escenario de arbitraje simultáneo se ejecuta sin bloqueos ni pérdidas de transacciones, lo que sugiere que el árbitro del DUT maneja correctamente las solicitudes concurrentes. El escenario de broadcast entrega correctamente la transacción a las tres terminales restantes, sin recepciones espurias ni faltantes. El escenario de dirección inválida no produce RX, lo cual es coherente con el comportamiento esperado del DUT ante destinos fuera de rango. El escenario de terminal acaparador completa la ráfaga de solicitudes desde un mismo origen sin que se observe bloqueo del árbitro. El escenario de autoenvío se clasifica correctamente como una transacción dirigida a la propia terminal, sin confundirse con una P2P.

IV-E. Observaciones sobre el escenario aleatorio

La secuencia soak de 15 transacciones se ejecuta sin reportar *mismatches* de payload, transacciones faltantes ni RX inesperadas. La mezcla de destinos válidos, inválidos y broadcast, junto con la variación de retardos y tamaños de payload, permite acumular 23 recepciones verificables distribuidas en los tres grupos de retardo observados en el histograma. La ausencia de coincidencias ambiguas indica que el checker distingue correctamente cada transacción y la asocia con su recepción correspondiente, incluso cuando varias transacciones comparten ventanas temporales cercanas.